Sliding Block Cipher Version 1 and the FPMQ2 Password File: Design, Repairs, and Open Questions

Ben P. Wise

21 October 2026

Abstract

The Sliding Block Cipher (SBC) is a block cipher of the author's design, first built in 2002 [14]. It encrypts the password file of `fpwdman`, the author's password manager. A review in October 2026 found no practical way to break SBC, but it found one structural weakness in the cipher and four weaknesses in the file format and the program around it. This article explains in plain language how the cipher and the file format work and why each part is there. It then describes the weaknesses and how they were repaired in SBC version 1 and in a new file format called FPMQ2, which `fpwdman` 4.0 writes. It gives pseudo-code for both, and it lists the attacks and analyses that remain to be done.

1 Introduction

A password manager keeps many secrets under one secret. The user remembers a single master passphrase; the program keeps every other password in a file encrypted under a key made from that passphrase. Anyone who copies the file can try to open it at leisure, on as many computers as they can afford. The design must therefore make two things hard: recovering the contents without the passphrase, and finding the passphrase by guessing.

`fpwdman` meets the first need with the Sliding Block Cipher (SBC), and the second with a standard key-derivation function. Section 2 explains both in plain language. Section 3 describes the weaknesses found in the review and how they were repaired. Section 4 gives pseudo-code for

the cipher and for the file format. Section 5 lists what remains to be studied. Appendix A records the measurements on which the article relies.

Three file formats appear throughout:

- the *legacy* format, written by the original FLTK program, which is read but no longer written;
- *FPMQ1*, written by the Qt program before version 4.0, which is also read but no longer written;
- *FPMQ2*, written by version 4.0 and later.

Opening a file in an older format and saving it converts it to FPMQ2.

2 How it works, and why

2.1 The cipher

SBC treats a block of data as a ring of K words of 32 bits each. The `fpwdman` settings use $K = 32$, so a block holds 1024 bits (128 bytes). The ring has no first or last word: the word before word 0 is word $K - 1$. A classic Feistel cipher (such as the US Data Encryption Standard, Blowfish, and many more) uses the same ring structure with $K = 2$. As there are only two elements in such a ring, they are conventionally referred to as the left and right blocks.

Encryption proceeds one *step* at a time. A step changes exactly one word, using the word just before it in the ring and one 32-bit *subkey*. Step 0 changes word 0, step 1 changes word 1, and so on around the ring. One pass around the ring is one *round*. `fpwdman` uses $R = 24$ rounds, that is $24 \times 32 = 768$ steps.

The change made at each step is

$$q(x) = (x + 1)(2x + 1) \bmod 2^{32}, \tag{2.1}$$

$$x_p \leftarrow x_p + \big(q(x_{p-1} \oplus s_i) \lll 7\big) \bmod 2^{32}, \tag{2.2}$$

where x_p is the word being changed, x_{p-1} the word before it, s_i the subkey for step i, $\oplus$ is bitwise exclusive-or, and $\lll 7$ rotates the 32 bits left by seven places. The function q (called `qTransform` in the code) is a quadratic that maps each 32-bit number to a different 32-bit number. It was adapted from the cipher RC6, and the added constant 1 removes the fixed points that RC6's version has.

The author adapted the quadratic transformation of RC6 to derive the general form of q as follows.

$$q(x) = (x + a)(nx + c) \bmod 2^m, \tag{2.3}$$

To ensure that $q(x) = q(y)$ if and only if $x = y$, we require that n is even positive, and c be odd positive.

To ensure that q has no fixed points , we require that a is positive.

Every 64 steps (two rounds) the words of the ring are rearranged by a permutation called `perm1`. It does not change any word; it only moves words to new places. The new order depends on the data: `perm1` combines all the words into one 32-bit number, by applying q to each and taking the exclusive-or of the results, and uses that number to choose the rearrangement.

A file is longer than one block, so the blocks are chained: each block is combined, by exclusive-or, with the encrypted block before it, and then encrypted. This is the standard cipher-block-chaining (CBC) mode. Decryption runs every step backward, from the last step to the first, subtracting where encryption added.

2.2 Why each part of the cipher is there

The step can always be undone.
Step (2.2) changes x_p by an amount that depends only on x_{p-1} and s_i, and x_{p-1} is not changed by that step. Whoever holds the key can therefore compute the same amount again and subtract it. This is the same idea as the Feistel ciphers, generalized from two words to a ring of any size, which is what lets SBC handle blocks of any length.

The step mixes bits nonlinearly.
Encryption that used only exclusive-or and addition could be undone by solving linear equations. The product in q and the mixing of exclusive-or with addition prevent that.

The rotation moves the high bits down.
In q, each output bit depends only on the input bits at the same place and below. Without the rotation, a change in a high bit could never reach the low bits. Rotating by seven places brings high bits to the bottom, so over many steps every bit influences every other.

The ring spreads a change to every word.
A change to word p alters the next step's change to word $p + 1$, which alters word $p + 2$,

and so on. Measured on random data, one round changes about a quarter of the 1024 bits of a block, and two rounds change half of them, which is what a random result would do (Appendix A).

The permutation moves words apart.
The step relates only neighbouring words. Moving words to new places between rounds makes words that were far apart into neighbours and vice versa.

Many rounds give a margin.
Most attacks on block ciphers grow harder with every round. Twenty-four rounds are far more than are needed to mix the bits, which leaves room against attacks not yet known.

Every step has its own subkey (version 1).
Section 3.1 explains why this matters.

2.3 The password file

Saving the password file in FPMQ2 takes the following stages.

1. Choose 16 random bytes, the *salt*.
2. Run PBKDF2 [12] on the passphrase and salt, with 600,000 repetitions of HMAC-SHA-256 [10, 5], to obtain a 32-byte *master key*.
3. Expand the master key with HKDF [11] into two independent keys: 64 bytes for the cipher and 32 bytes for the authentication code.
4. Place 16 more random bytes, the *nonce*, before the list of entries, followed by its length; pad the whole to a multiple of 128 bytes.
5. Encrypt it with SBC version 1 in CBC mode.
6. Compute an authentication code (HMAC-SHA-256) over the file's header and the encrypted text, and write the header, the code, and the encrypted text.

Opening the file repeats stages 2 and 3, recomputes the authentication code, and decrypts only if it matches.

2.4 Why each part of the file is there

The 600,000 repetitions make guessing slow.
The user pays the cost once when opening the file, about a second. An attacker pays it for

every passphrase tried. This cost is the main protection for a stolen file, and it must not be reduced to make the program feel faster.

The salt prevents precomputation.
Because every file has its own salt, an attacker cannot compute a table of common passphrases and their keys once and use it against every file. Each guess is good for one file only. The salt need not be secret, and it cannot be: the program needs it to make the key, and the key is needed to decrypt anything. It is therefore written in the clear, as in every standard format.

The nonce makes every encryption different.
The random bytes at the front of the plaintext change the whole encrypted file, because CBC carries the change from the first block into every later one. Since the salt is also new at every save, the key changes too; the nonce is a second safeguard.

The authentication code detects a wrong passphrase and any tampering.
The code is checked before anything is decrypted. A wrong passphrase gives a wrong key, and therefore a code that does not match. A file changed by anyone without the passphrase also fails the check. Checking the code before decrypting ("encrypt-then-MAC") is the order that is known to be safe [1].

The cipher and authentication keys are separate.
Using one key for two purposes invites unforeseen interactions between them.

3 Weaknesses found and repaired

Table 1 summarizes the weaknesses. The subsections explain each.

3.1 Identical rounds and the slide attack

In version 0 of SBC, step i used the subkey $s_i = q(k_{i \bmod 16})$, where $k_0, \ldots, k_{15}$ are the 16 words of the key. The subkeys therefore repeat every 16 steps, the word positions repeat every 32, and `perm1`, which uses no key, runs every 64. Between one permutation and the next, the cipher applied exactly the same function. A test confirmed that the state after each permutation is that same function applied to the state after the one before (Appendix A).

Weakness	Effect	Repair
Every round of SBC the same	Slide attacks, whose cost does not depend on the number of rounds	SBC version 1: a different subkey for every step, made with HKDF
Iteration count read from the file without a check	A crafted file could freeze the program for about 40 minutes	Only 600,000 is accepted, checked before any key is made
Three PBKDF2 runs per open	The user paid three times what an attacker pays	FPMQ2: one PBKDF2 run, expanded with HKDF
Header outside the authentication code	Header could be changed without detection	FPMQ2: the code covers the header
A backup copy kept at every save	The previous file survived in its old format and under its old passphrase	Version 4.0 keeps no backup

Table 1: Weaknesses found in the October 2026 review and their repairs.

A cipher built of identical keyed rounds is open to the *slide attack* of Biryukov and Wagner [3, 4]. Let F be one round under a fixed key, and let the cipher be $E = F^r$, that is F applied r times. Then

$$E(F(P)) = F^{r+1}(P) \tag{3.1}$$
$$= F(E(P)). \tag{3.2}$$

Suppose an attacker holds two plaintexts P and P' with $P' = F(P)$, a *slid pair*. By (3.2) their ciphertexts satisfy $C' = F(C)$. The attacker then has two inputs and outputs of a single round, (P, P') and (C, C'). A single round is far weaker than the whole cipher, so the key can be sought from one round instead of 24.

The attacker cannot compute F, and so cannot make a slid pair on purpose, but must find one by chance among many known plaintexts and ciphertexts. By the birthday argument this takes about $2^{n/2}$ pairs for an n-bit block, and the number does not depend on r. Adding rounds does not raise it.

For SBC, with $n = 1024$, the cost is about 2^{512} known blocks under one key, which no one can collect; `fpwdman`, besides, uses a new key at every save. The weakness is therefore one of principle, not of practice. It is still a design defect, because it bypasses the number of rounds, which is the cipher's main margin of safety.

The repair. Version 1 makes a separate subkey for every step from the key, by HKDF:

$$(s_0, s_1, \ldots, s_{RK-1}) = \mathrm{HKDF}\big(key,\ salt = \text{empty},\ info = \text{“SBC v1 subkeys”} \,\|\, K \,\|\, R,\ L = 4RK \text{ bytes}\big), \tag{3.3}$$

where $\|$ joins byte strings and K and R are written as four-byte numbers. Each run of four output bytes is one subkey. Nothing else in the cipher changes. No two rounds now use the same subkeys, and there is no simple rule that links the subkeys of one key to those of another. Writing K and R into the label keeps the schedules of different shapes unrelated: without them, the 24-round schedule would be the start of the 25-round one.

A repair that does not work. An obvious alternative is to add a different constant at each step, for example

$$s_i = q\big(k_{i \bmod 16} + (i+1)\varphi\big) \bmod 2^{32}, \tag{3.4}$$

$$\varphi = 9\mathrm{E}3779\mathrm{B}9_{16}. \tag{3.5}$$

Every step then has its own subkey, but the subkeys are related in a simple way. Take a second key k' whose every word is larger by 64φ. Step i under k' then uses exactly the subkey of step $i + 64$ under k. Sixty-four steps is a whole number of rounds and also matches the permutation schedule, so the two keys give the same cipher shifted by 64 steps, and the slide attack returns for pairs of related keys. Any constant that grows regularly with the step number has this flaw. HKDF has no such regularity. A milder alternative that keeps the key word in every step is

$$s_i = q\big(k_{i \bmod N} \oplus c_i\big), \tag{3.6}$$

with c_i the first 32 bits of SHA-256 of “SBC-RC” and i. It also defeats the slide attack, but it leaves the subkeys simply related to the key, so version 1 uses HKDF.

3.2 The unchecked iteration count

The header of an FPMQ1 file states the number of PBKDF2 repetitions, and the program used that number as written, checking only that it was positive. The number cannot be authenticated before it is used, because the authentication key itself comes out of PBKDF2. A crafted file giving 2,147,483,647 repetitions would have made the program compute about 6.4 billion HMACs, roughly 40 minutes in which the window would not respond. Nothing secret would leak, so the defect was a nuisance rather than a break. Both formats now accept only 600,000, and the check is made before any key is derived.

3.3 The threefold cost of key derivation

PBKDF2 produces 32 bytes for each full run of its repetitions. FPMQ1 asked for 96 bytes, so every open ran PBKDF2 three times: 1.8 million HMACs. But the authentication key was the third 32-byte block by itself, so an attacker testing a guess needed to compute only that block, 600,000 HMACs. The extra 1.2 million gave the user delay and no protection. FPMQ2 runs PBKDF2 once for 32 bytes and expands them with HKDF, which costs two HMACs. Saving and opening a file now takes about a third as long as before (Appendix A), with the attacker's cost unchanged.

3.4 The unauthenticated header

FPMQ1 computed its authentication code over the encrypted text only. Changing the salt or the repetition count would still have failed, because those change the key, but the version name and the other header text were not protected. FPMQ2 computes the code over both header lines and the encrypted text, and it rejects any header that differs from what the program writes. In particular, an FPMQ2 file relabelled as FPMQ1 is refused, so it cannot be steered onto the older reading path.

3.5 The backup copy

Before version 4.0, every save first copied the existing file to a backup with the extension `.bak`. The backup kept the previous file exactly, in its old format and under its old passphrase. Two cases made that a weakness. After a legacy file was converted, the backup was the legacy file itself, which allows fast guessing of a passphrase that also opens the new file. After a change of master passphrase, the backup could still be opened with the old one, which defeats the purpose of changing it. Version 4.0 keeps no backup. A failed save is still harmless: the program writes the new file under a temporary name and replaces the old one only when the new one is complete, so an interrupted save leaves the old file as it was.

3.6 Earlier repairs

For completeness: the legacy format of the FLTK program had no key-derivation function. The passphrase was scrambled into the key by a reversible function, with no salt and no delay, and a wrong guess could be recognized after decrypting a single block. Guessing was therefore millions

of times cheaper than for FPMQ1 or FPMQ2. The Qt program corrected this with FPMQ1 before the present review, and it now only reads the legacy format.

4 Pseudo-code

Throughout, words are 32 bits; $+$ and $-$ are modulo 2^{32}; $\oplus$ is exclusive-or; $\lll$ is left rotation. The cipher's parameters are the block size K (32), the key size N (16 words), the rounds R (24), and the permutation interval $M = 2K$ (64).

4.1 The basic SBC

Algorithms 1 to 4 give the cipher: its primitive functions, its two key schedules, the encryption and decryption of one block, and the chaining of blocks. Two points need comment. First, in Perm1 the value σ does not depend on the order of the words, so decryption can compute it from the permuted block, and so undo the permutation. Second, the chaining starts from zero rather than from a random value, but the first 16 bytes of every plaintext are random (Algorithm 5), which serves the same purpose.

4.2 FPWDMAN

Algorithm 5 saves the password file and Algorithm 6 opens it. A legacy file is recognized by its first line, which holds two numbers, and is decoded by a separate reader that reproduces the FLTK program exactly. Saving any file, whatever its format, calls Save.

Algorithm 1 The primitive functions

function $\textsc{Q}(x)$
  return $(x+1)(2x+1)$ ▷ modulo 2^{32}
end function
function $\textsc{fold}(j)$ ▷ stretch and fold of the indices $0 \dots K-1$
  if $2j \le K-1$ **then**
    return $2j$
  else
    return $2(K-j)-1$
  end if
end function
function $\textsc{Perm1}(x,$ *forward*$)$
  $\sigma \leftarrow q(x_0) \oplus q(x_1) \oplus \cdots \oplus q(x_{K-1})$ ▷ does not depend on the order
  for $t \leftarrow 0$ **to** 7 **do**
    $\sigma_t \leftarrow \sigma \lll 7t$
  end for
  if *forward* **then**
    for $t \leftarrow 0$ **to** 7 **do**
      for every j: $y_{\textsc{fold}((j+\sigma_t) \bmod K)} \leftarrow x_j$; then $x \leftarrow y$
    end for
  else ▷ undo the eight moves in reverse order
    for $t \leftarrow 7$ **down to** 0 **do**
      for every j: $y_j \leftarrow x_{\textsc{fold}((j+\sigma_t) \bmod K)}$; then $x \leftarrow y$
    end for
  end if
  return x
end function

Algorithm 2 Key setup

function KeySetupV0($k_0, \ldots, k_{N-1}$) ▷ legacy and FPMQ1 only
  for $i \leftarrow 0$ **to** $RK - 1$ **do**
    $s_i \leftarrow q(k_{i \bmod N})$ ▷ repeats every N steps
  end for
  return s
end function
function KeySetupV1(key) ▷ any non-empty byte string
  $info \leftarrow$ "SBC v1 subkeys" $\|$ BE32(K) $\|$ BE32(R)
  $b \leftarrow$ HKDF-SHA-256(key, empty salt, $info$, $4RK$)
  for $i \leftarrow 0$ **to** $RK - 1$ **do**
    $s_i \leftarrow$ bytes $4i \ldots 4i + 3$ of b, read most significant first
  end for
  return s ▷ $RK \leq 2040$, the limit of HKDF
end function

Algorithm 3 Encrypting and decrypting one block

function EncipherBlock($x_0, \ldots, x_{K-1}$; s)
  for $i \leftarrow 0$ **to** $RK - 1$ **do**
    $p \leftarrow i \bmod K$
    $x_p \leftarrow x_p + \big(q(x_{(p-1) \bmod K} \oplus s_i) \lll 7\big)$
    if $i > 0$ **and** $i \bmod M = 0$ **then**
      $x \leftarrow$ Perm1(x, true)
    end if
  end for
  return x
end function
function DecipherBlock($x_0, \ldots, x_{K-1}$; s)
  for $i \leftarrow RK - 1$ **down to** 0 **do**
    if $i > 0$ **and** $i \bmod M = 0$ **then**
      $x \leftarrow$ Perm1(x, false)
    end if
    $p \leftarrow i \bmod K$
    $x_p \leftarrow x_p - \big(q(x_{(p-1) \bmod K} \oplus s_i) \lll 7\big)$
  end for
  return x
end function

Algorithm 4 Chaining blocks (CBC with a zero starting value)

function EncipherCBC($P_0, \ldots, P_{m-1}$; s)
    $C_{-1} \leftarrow 0$
    for $b \leftarrow 0$ **to** $m - 1$ **do**
        $C_b \leftarrow$ EncipherBlock($P_b \oplus C_{b-1}; s$)
    end for
    return $C_0, \ldots, C_{m-1}$
end function
function DecipherCBC($C_0, \ldots, C_{m-1}$; s)
    $C_{-1} \leftarrow 0$
    for $b \leftarrow 0$ **to** $m - 1$ **do**
        $P_b \leftarrow$ DecipherBlock($C_b; s$) $\oplus C_{b-1}$
    end for
    return $P_0, \ldots, P_{m-1}$
end function

Algorithm 5 Saving the password file (FPMQ2)

function Save(*passphrase*, *entries*)
    xml ← the entries written as XML
    salt ← 16 random bytes
    master ← PBKDF2-HMAC-SHA-256(*passphrase*, *salt*, 600000, 32)
    $k_c \leftarrow$ HKDF-Expand(*master*, "FPMQ2 sbc", 64)
    $k_m \leftarrow$ HKDF-Expand(*master*, "FPMQ2 mac", 32)
    $F \leftarrow$ 16 random bytes ‖ LE32(length of *xml*) ‖ *xml*
    pad F with bytes FF_{16} to a multiple of 128 bytes
    $C \leftarrow$ EncipherCBC(F; KeySetupV1(k_c))
    $H \leftarrow$ "FPMQ2\n" ‖ "kdf=pbkdf2-sha256; iters=600000; salt=" ‖ Base64(*salt*) ‖ "\n"
    tag ← HMAC-SHA-256($k_m, H \,\|\, C$)
    write H, then "mac=hmac-sha256; tag=" ‖ Base64(*tag*), then Base64(C) in lines of 64 characters
    erase *master*, k_c, k_m, F and *xml* from memory
end function

Algorithm 6 Opening the password file (FPMQ1 or FPMQ2)

function OPEN(*file*, *passphrase*)
  read the version line, the salt, the repetition count c, the tag and C
  if the version is neither FPMQ1 nor FPMQ2 **then**
    fail: corrupt file
  end if
  if $c \neq 600000$ **then** ▷ checked before any key is made
    fail: corrupt file
  end if
  if FPMQ2 **and** the header differs from what SAVE writes **then**
    fail: corrupt file
  end if
  if FPMQ2 **then**
    derive k_c, k_m as in SAVE; $data \leftarrow H \,\|\, C$; $schedule \leftarrow$ KEYSETUPV1
  else ▷ FPMQ1
    $d \leftarrow$ PBKDF2-HMAC-SHA-256$(passphrase, salt, 600000, 96)$
    $k_c \leftarrow d_{0\ldots63}$; $k_m \leftarrow d_{64\ldots95}$; $data \leftarrow C$; $schedule \leftarrow$ KEYSETUPV0
  end if
  if HMAC-SHA-256$(k_m, data) \neq tag$ **then** ▷ compared in constant time
    fail: wrong passphrase, or the file was changed
  end if
  $F \leftarrow$ DECIPHERCBC$(C;\ schedule(k_c))$
  skip 16 bytes; read the length; **return** the entries parsed from the XML
end function

5 Remaining attacks and analyses

The review established the weaknesses above and measured some properties (Appendix A). It did not attempt the analyses below, which a cipher must survive before it can be trusted for important data.

5.1 Analyses of the cipher

1. **Strength of a single round group.** The slide attack pays only if 64 steps are much weaker than 768. That is usual, but it has not been shown for SBC. How much work recovers the key from one input and output of 64 steps?

2. **Differential cryptanalysis** [2]. Search for differences in the input that pass through a few rounds with high probability, and from them estimate how many rounds an attack could reach. The quadratic q is a *T-function* [7]: each output bit depends only on the input bits at the same place and below. Differences in the low bits therefore propagate predictably within q, and only the rotation and the addition carry them upward.

3. **Linear cryptanalysis** [8]. The two lowest output bits of q are linear: bit 0 is the complement of input bit 0, and bit 1 is a linear function of input bits 0 and 1. In version 0 of the 64-bit library a similar property made four bits of every step exactly linear. In the 32-bit cipher the addition in step (2.2) hides them behind carries. Whether useful linear approximations survive several rounds is not known.

4. **Reduced-round attacks.** Attack versions of SBC with 1, 2, 3, and more rounds, to learn how large the safety margin of 24 rounds actually is. This is the standard way of measuring a new cipher.

5. **The permutation.** `perm1` uses no key, and its choice depends on the exclusive-or of q applied to every word, which does not depend on their order. It reads only 30 of those 32 bits. It produces a great variety of rearrangements (65,535 different ones from 65,536 random blocks), so it mixes well, but anyone who knows the state at that point can compute it. Does it add strength, or only spreading? Are there blocks it leaves in a predictable order? A keyed permutation would be worth studying as a version 2.

6. **Algebraic attacks.** Each step is a polynomial of low degree over the integers modulo 2^{32}, mixed with exclusive-or. Methods that solve systems of such equations should be tried on reduced rounds.

7. **Statistical tests.** Run the output of SBC in counter mode through standard test batteries for random numbers, such as NIST SP 800-22 or PractRand, at full and at reduced rounds.

8. **Related keys under version 1.** With subkeys from HKDF, no relation between keys is known to carry over to the subkeys. In `fpwdman` the keys come from PBKDF2 and cannot be chosen by an attacker, so this matters mainly for SBC as a standalone cipher.

5.2 Analyses of the file format and the program

9. **Old copies of the file.** A legacy copy of the file, under the same passphrase as the current one, lets an attacker guess that passphrase quickly and then open the current file. Version 4.0 makes no backup copy, but copies can survive elsewhere: a `.bak` written by an earlier version, the user's own backups, synchronized folders, and snapshots kept by the operating system. After converting, the user should delete such copies where possible and then change the master passphrase, which makes any copy that cannot be found harmless to the current file.

10. **Memory-hard key derivation.** PBKDF2 can be computed cheaply in parallel on graphics processors. Argon2id [13], which also requires much memory, is now preferred [9]. A future format could use it.

11. **Unbounded inputs.** The program does not yet limit the size of a file it will read, nor the size a legacy file may claim for its decompressed contents. A crafted file could therefore demand a very large allocation. Limits should be set before any decoding.

12. **The length of the file.** Encryption hides the contents but not their approximate size, which reveals roughly how many entries the file holds. Padding to a fixed size would hide it.

13. **Timing.** `perm1` writes words to places that depend on the data. On a shared computer, the timing of memory caches could in principle reveal something about those places. This matters only to an attacker running programs on the same machine at the same time.

14. **Secrets left in memory.** The program erases keys and plaintext after use, but Qt's strings are copied freely, so some copies cannot be reached and erased. The clipboard, the operating system's swap file and crash dumps can also hold secrets.

15. **Comparison with standard ciphers.** SBC has had one reviewer. The standard ciphers, such as AES-256-GCM [6] and XChaCha20-Poly1305, have had decades of study by many. Two earlier reviews of `fpwdman` recommended writing new files with one of them and

reading SBC files only. That remains the most conservative choice; the work in this article is the alternative of strengthening SBC and inviting its analysis.

A Measurements

All measurements below used keys and data generated at random for the purpose, or fixed test vectors. A Python implementation of SBC version 0 was first checked bit for bit against the compiled C++ program on a two-block test vector; the version 1 test vector in the program's tests was computed with the same Python implementation, using Python's own HMAC and SHA-256.

Spreading of changes. Mean number of the 1024 output bits changed when one bit of the input is changed, over 100 random keys and blocks (a random result would change 512):

Rounds	Mean bits changed	Fewest in any trial
1	249.2	14
2	512.2	481
3	511.9	476
4	513.4	470
24	513.6	475

Changing one bit of the key changed 510.5 bits on average after 24 rounds.

Linearity of q. The algebraic degree of output bits 0 to 5 of q, as functions of the input bits at the same place and below, is 1, 1, 2, 3, 4, 5. Only the two lowest bits are linear.

The permutation. `perm1` reads 30 of the 32 bits of its combined value (bits 9 and 10 are never used). Among 65,536 random values it produced 65,535 different rearrangements.

Identical rounds in version 0. In all 10 checks, the state after permutation $m + 1$ equalled one fixed 64-step function applied to the state after permutation m. Under version 1, no two of the 24 rounds have the same subkeys, while under version 0 all 24 do; the program's tests check both.

Speed. One save and one open, in the program's debug build: 6.6 seconds for FPMQ1 and 2.3 seconds for FPMQ2.

Tests. Version 4.0 passes 36 tests of the cipher and key derivation, including the published test vectors for SHA-256, HMAC-SHA-256, PBKDF2 and HKDF, and 23 tests of the file formats, including the conversion of legacy and FPMQ1 files and the rejection of altered FPMQ2 files.

References

[1] M. Bellare and C. Namprempre, "Authenticated encryption: relations among notions and analysis of the generic composition paradigm," *Advances in Cryptology – ASIACRYPT 2000*, LNCS 1976, Springer, 2000.

[2] E. Biham and A. Shamir, "Differential cryptanalysis of DES-like cryptosystems," *Journal of Cryptology*, vol. 4, no. 1, 1991.

[3] A. Biryukov and D. Wagner, "Slide attacks," *Fast Software Encryption (FSE '99)*, LNCS 1636, Springer, 1999.

[4] A. Biryukov and D. Wagner, "Advanced slide attacks," *Advances in Cryptology – EUROCRYPT 2000*, LNCS 1807, Springer, 2000.

[5] National Institute of Standards and Technology, *Secure Hash Standard (SHS)*, FIPS PUB 180-4, 2015.

[6] National Institute of Standards and Technology, *Advanced Encryption Standard (AES)*, FIPS PUB 197, 2001.

[7] A. Klimov and A. Shamir, "A new class of invertible mappings," *Cryptographic Hardware and Embedded Systems (CHES 2002)*, LNCS 2523, Springer, 2002.

[8] M. Matsui, "Linear cryptanalysis method for DES cipher," *Advances in Cryptology – EUROCRYPT '93*, LNCS 765, Springer, 1994.

[9] OWASP Foundation, *Password Storage Cheat Sheet*, https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html.

[10] H. Krawczyk, M. Bellare and R. Canetti, *HMAC: Keyed-Hashing for Message Authentication*, RFC 2104, 1997.

[11] H. Krawczyk and P. Eronen, *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*, RFC 5869, 2010.

[12] K. Moriarty, B. Kaliski and A. Rusch, *PKCS #5: Password-Based Cryptography Specification Version 2.1*, RFC 8018, 2017.

[13] A. Biryukov, D. Dinu, D. Khovratovich and S. Josefsson, *Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*, RFC 9106, 2021.

[14] B. P. Wise, *Sliding Block Ciphers*, draft, 2006.